

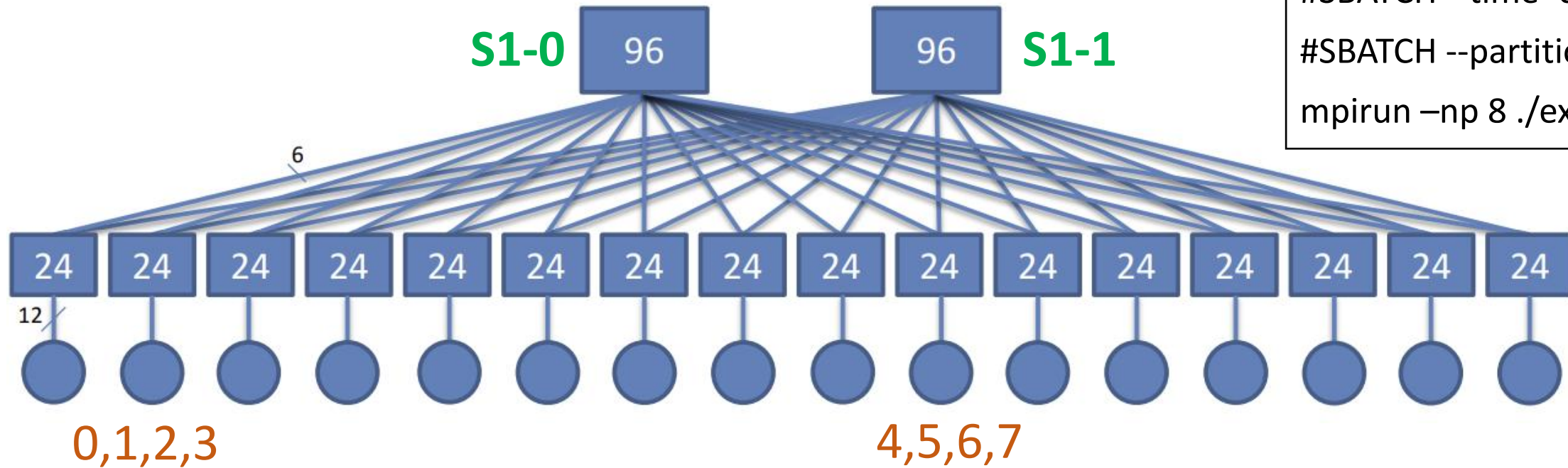
Communication Cost

Lecture 10

February 9, 2026

Rank Mapping on Supercomputers

```
#SBATCH -N 2
#SBATCH --ntasks-per-node=4
#SBATCH --error=job.%J.err
#SBATCH --output=job.%J.out
#SBATCH --time=00:10:00
#SBATCH --partition=standard
mpirun -np 8 ./exe <args>
```



32 processes on 2 nodes, ppn=16

Output (sample hello MPI code)

Hello from rank 0 out of 32 on system rdcn26

Hello from rank 1 out of 32 on system rdcn26

....

Hello from rank 15 out of 32 on system rdcn26

Hello from rank 16 out of 32 on system rdcn27

Hello from rank 17 out of 32 on system rdcn27

....

Hello from rank 30 out of 32 on system rdcn27

Hello from rank 31 out of 32 on system rdcn27

```
#!/bin/bash
```

```
#SBATCH --job-name=mpi_hello
```

```
#SBATCH -N 2
```

```
#SBATCH --ntasks-per-node=16
```

```
#SBATCH --output=mpi_hello_%j.out
```

```
#SBATCH --error=mpi_hello_%j.err
```

```
#SBATCH --partition=cpu
```

```
#SBATCH --time=00:10:00
```

```
module load compiler/oneapi-2024/mpi
```

```
mpirun -np 32 ./hello
```

64 processes on 2 nodes, ppn=16

Hello from rank 0 out of 64 on system rdcn26

Hello from rank 1 out of 64 on system rdcn26

....

Hello from rank 14 out of 64 on system rdcn26

Hello from rank 15 out of 64 on system rdcn26

Hello from rank 16 out of 64 on system rdcn27

Hello from rank 17 out of 64 on system rdcn27

....

Hello from rank 30 out of 64 on system rdcn27

Hello from rank 31 out of 64 on system rdcn27

Hello from rank 32 out of 64 on system rdcn26

Hello from rank 33 out of 64 on system rdcn26

....

Hello from rank 46 out of 64 on system rdcn26

Hello from rank 47 out of 64 on system rdcn26

Hello from rank 48 out of 64 on system rdcn27

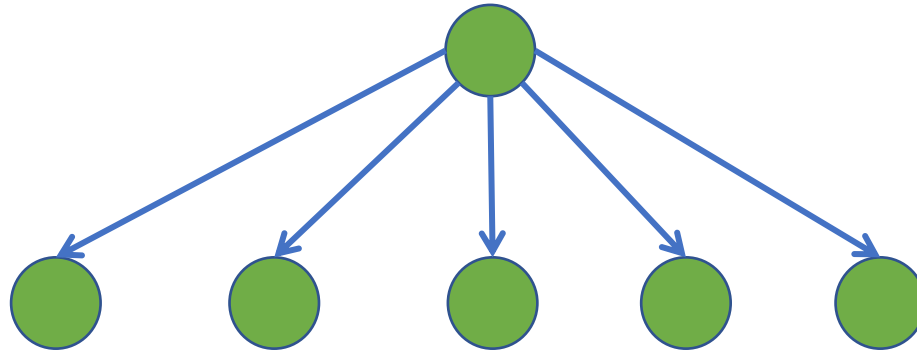
Hello from rank 49 out of 64 on system rdcn27

....

Hello from rank 62 out of 64 on system rdcn27

Hello from rank 63 out of 64 on system rdcn27

Timing Analysis for 1-to-many Sends



I Communication volume = 10000 integers to 29 ranks in 0.02 s

➔ Throughput/bandwidth achieved = $10000 * 4 * 29 \text{ bytes} / 0.02 \text{ s} = 55.3 \text{ Mbps}$

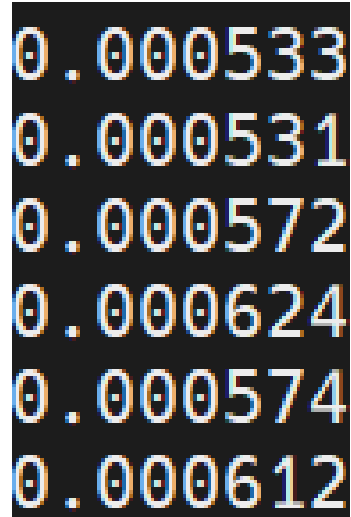
II Communication volume = 1000000 integers to 29 ranks in 0.8 s

➔ Throughput/bandwidth achieved = $1000000 * 4 * 29 \text{ bytes} / 0.8 \text{ s} = 138.3 \text{ Mbps}$

Note: This is not the network bandwidth

Timing MPI_Send (2 nodes)

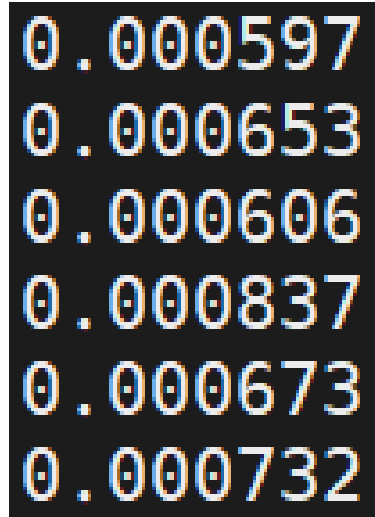
```
for i in `seq 1 3` ; do mpirun -np 2 -hosts csews2,csews16 ./timingSend X ; done  
// X integers are being sent from one node to another (three times)  
// Times from both ranks are printed
```



0.000533	0.000531
0.000572	0.000624
0.000574	0.000612

X = 4

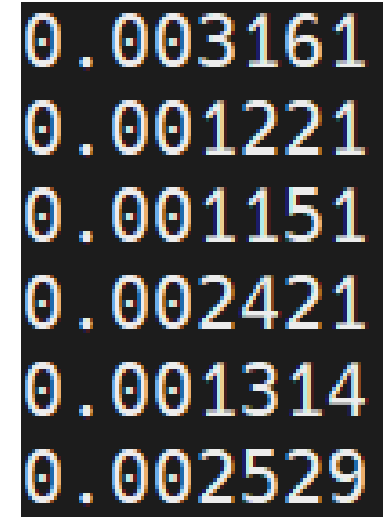
0.03 MBps



0.000597	0.000653
0.000606	0.000837
0.000673	0.000732

X = 400

2.54 MBps



0.003161	0.001221
0.001151	0.002421
0.001314	0.002529

X = 40000

76.29 MBps

Note the gradual increase in achieved bandwidth with increase in data size

Effective Bandwidth (2 nodes on csews) Run 1

100 B ~ 24 MB/s

500 B ~ 79 MB/s

1 KB ~ 97 MB/s

100 KB ~ 109 MB/s

1 MB ~ 108 MB/s

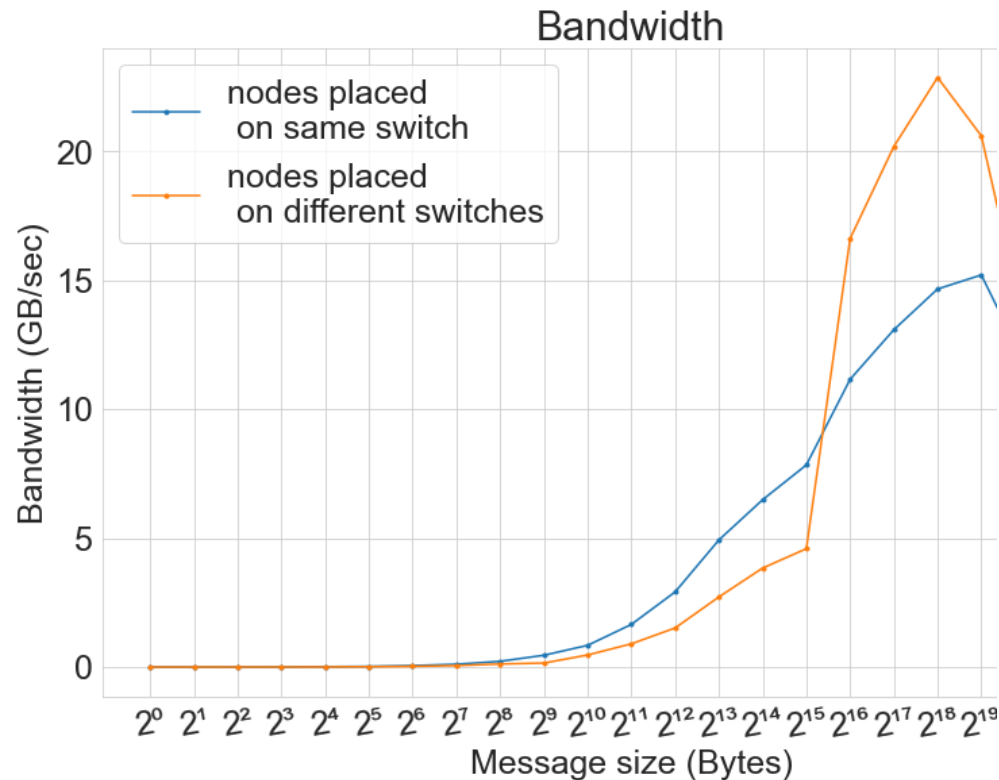
10 MB ~ 111 MB/s

Effective Bandwidth (2 nodes on csews) Run 2

Data Size	Time (s)	Bandwidth (MB/s)	Data Size	Time (s)	Bandwidth (MB/s)
1 KB	0.0005	1.95	256 KB	0.0032	78.13
2 KB	0.0005	3.91	512 KB	0.0054	92.59
4 KB	0.0005	7.81	1 MB	0.0099	101.01
8 KB	0.0005	15.62	2 MB	0.0190	105.26
16 KB	0.0005	31.24	4 MB	0.0370	108.10
32 KB	0.0007	44.65	8 MB	0.0740	108.10
64 KB	0.0012	52.08	16 MB	0.1460	109.58
128 KB	0.0024	52.08	32 MB	0.2902	110.26

Note: This is as per measurement done on a random day on random 2 nodes

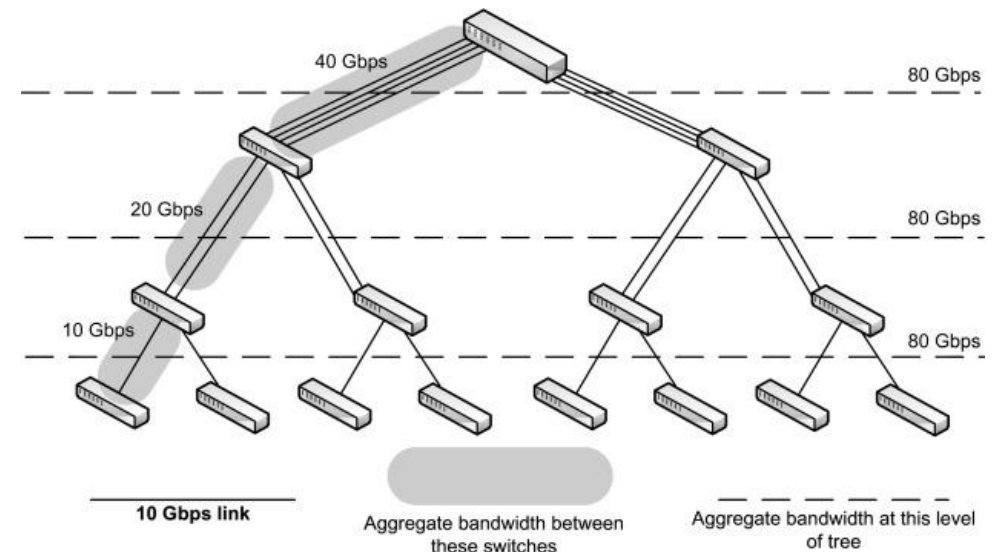
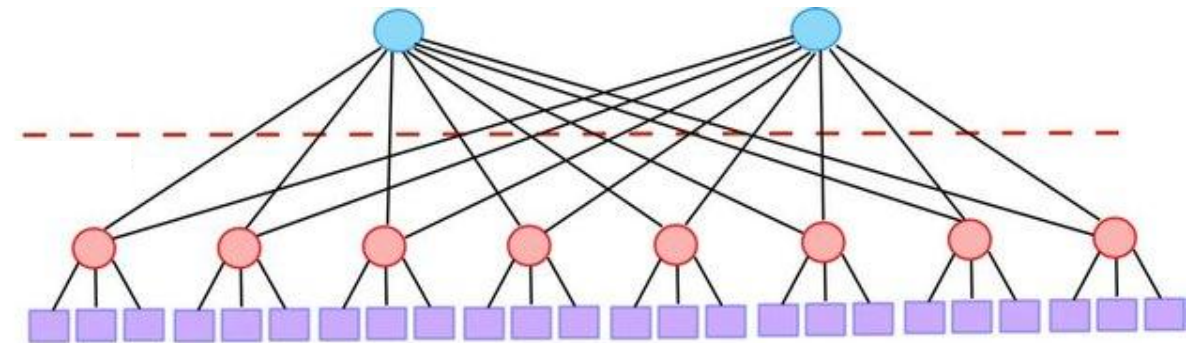
Bandwidth (PARAM Sanganak)



All node-pairs communicate. Similar performance in case of latency-bound region; worse performance due to congestion in case of same switch case when the data size is large (note the cross-over point after 2^{15} bytes)

Image Credit: S. Aggarwal, 2021

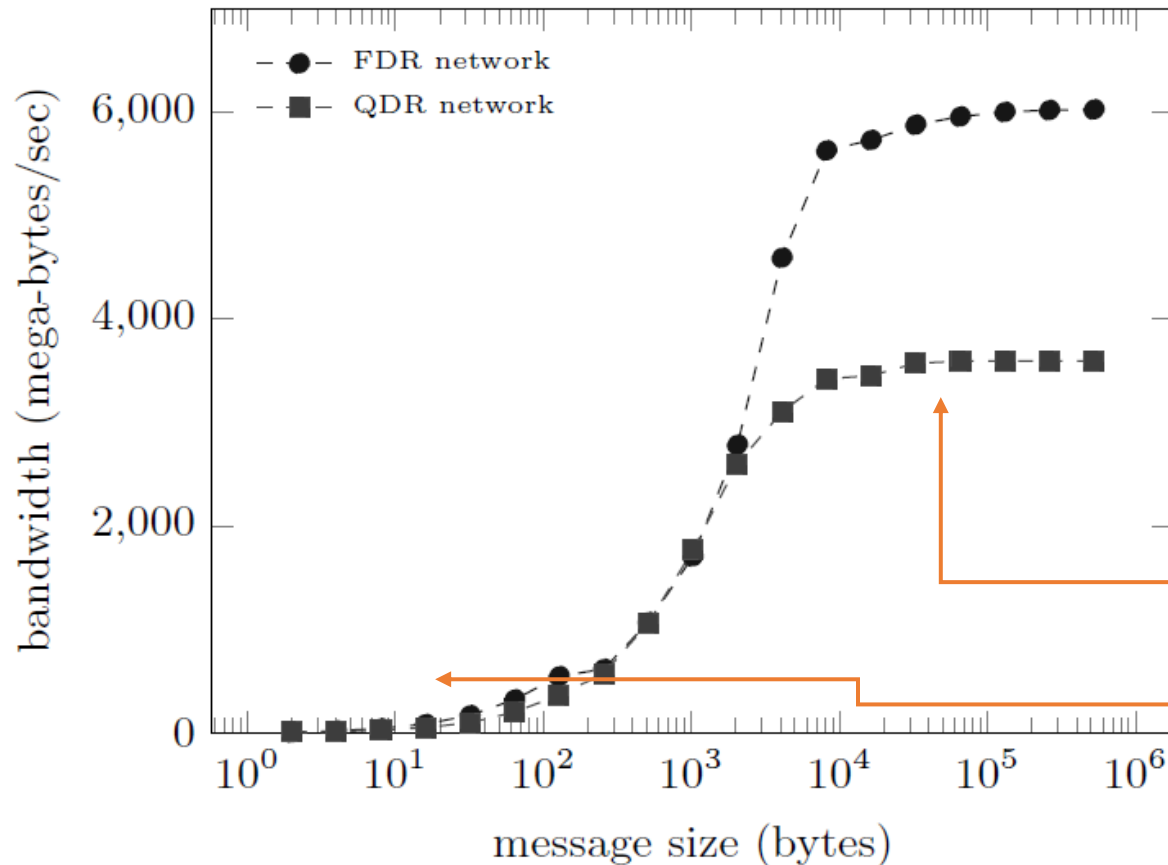
Fat-tree topology [researchgate.com]



Fat-tree topology [www.sciencedirect.com]

Effective Bandwidth

QDR Infiniband speed (bandwidth): 10x4 Gbps
FDR Infiniband speed (bandwidth): 14x4 Gbps

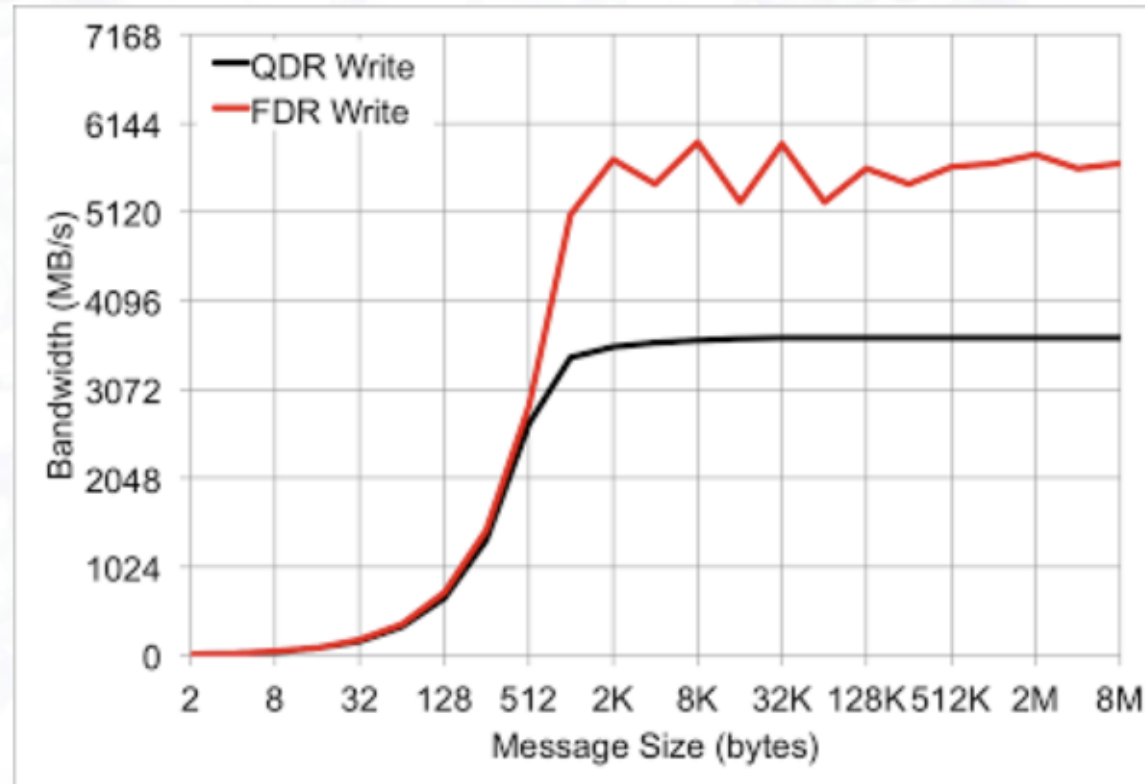


6000 MB/s
= $6000/1024$ GB/s
= $5.85 * 8$ Gbps
= 46.87 Gbps
(83.7% efficiency)

Source: Barthels et al., SIGMOD'15

Latency-bound: The leftmost (almost) flat region
Bandwidth-bound: The rightmost (almost) flat region

QDR vs. FDR



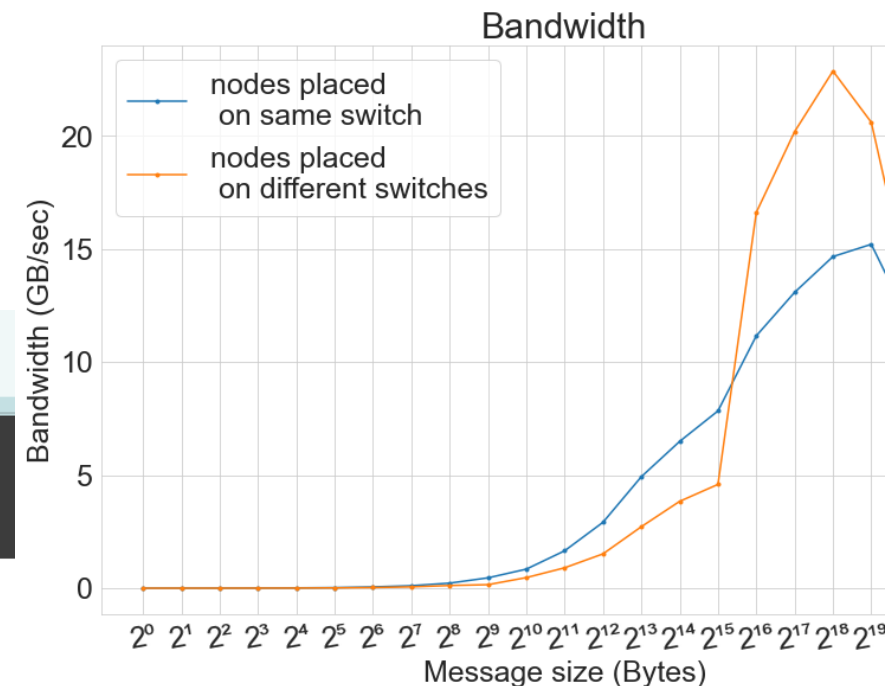
Source: G. Lockwood, NERSC, 2013

System Architecture and Configuration

System Hardware Specifications

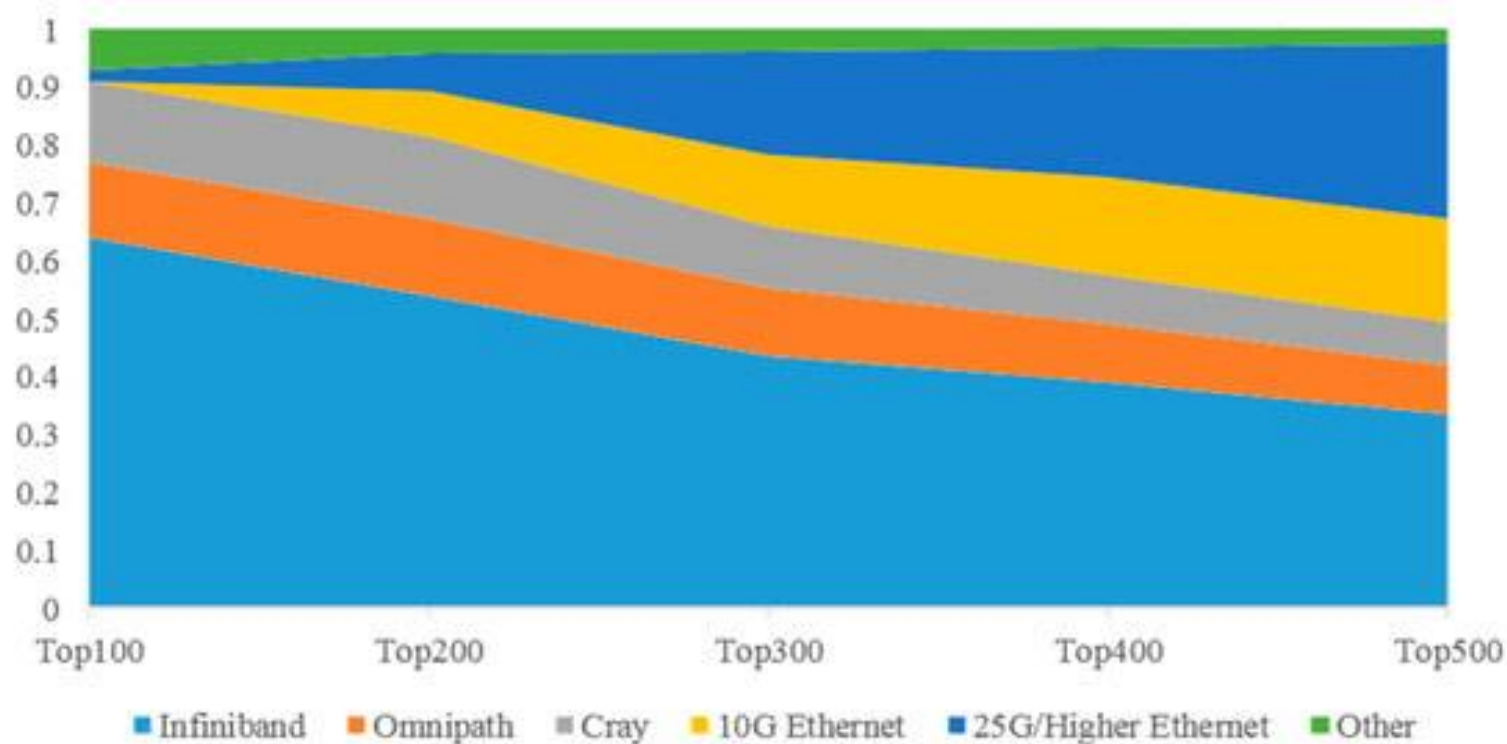
PARAM Sanganak systems are based on Intel Xeon Platinum 8268, NVIDIA Tesla V100 with total peak performance of 1.6 PFLOPS. The cluster consists of compute nodes connected with Mellanox (HDR) InfiniBand interconnect network. The system uses the Lustre parallel file system.

200 Gbps



~93% efficiency

Network Distribution in Top500 Supercomputers



Source: A Survey of High-Performance Interconnection Networks in High-Performance Computer Systems, Electronics 2022

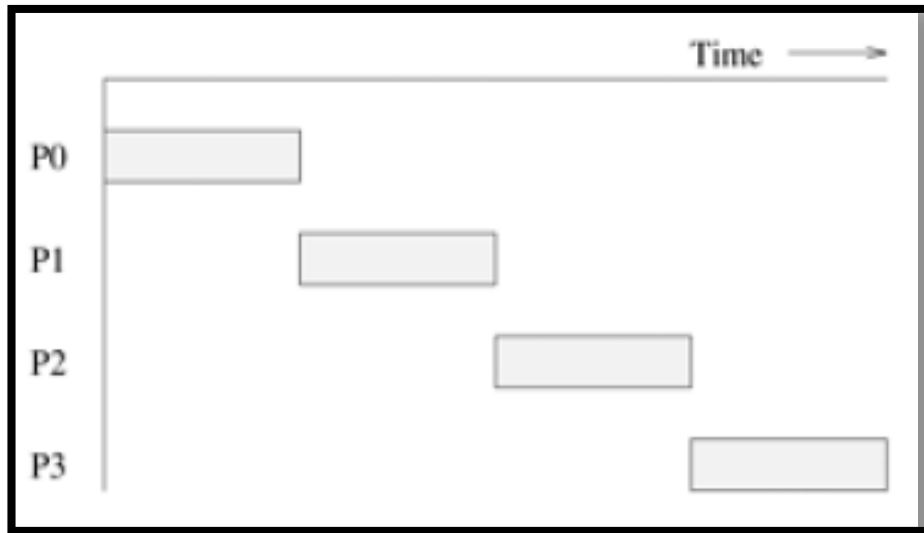
Communication Cost

- Startup time (t_s)
 - Time required to handle a message at the sending and receiving nodes
- Per-hop time (t_h)
 - Node latency
- Per-word transfer time (t_w) / Bandwidth (B)
 - Channel capacity

Transfer time of n words on l links = ?

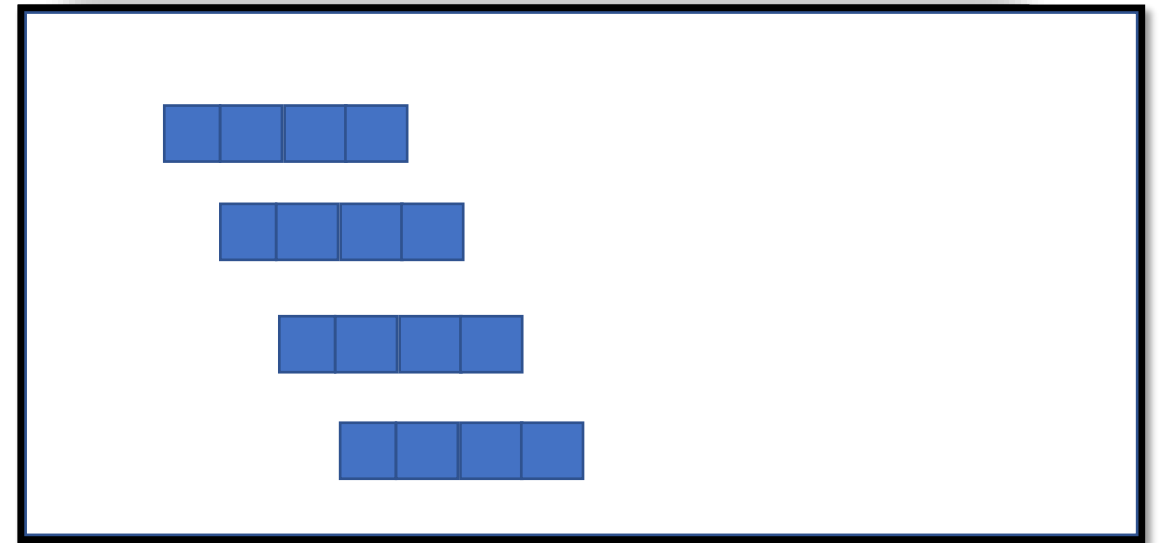
Routing

$$t_s + l(t_h + n/B)$$



Store-and-forward

$$t_s + l.t_h + n/B$$



Cut-through

Reference: Chapter 2 of "Introduction to Parallel Computing", GGKK

(Optional) Virtual cut-through: A new computer communication switching technique, Computer Networks, 1976

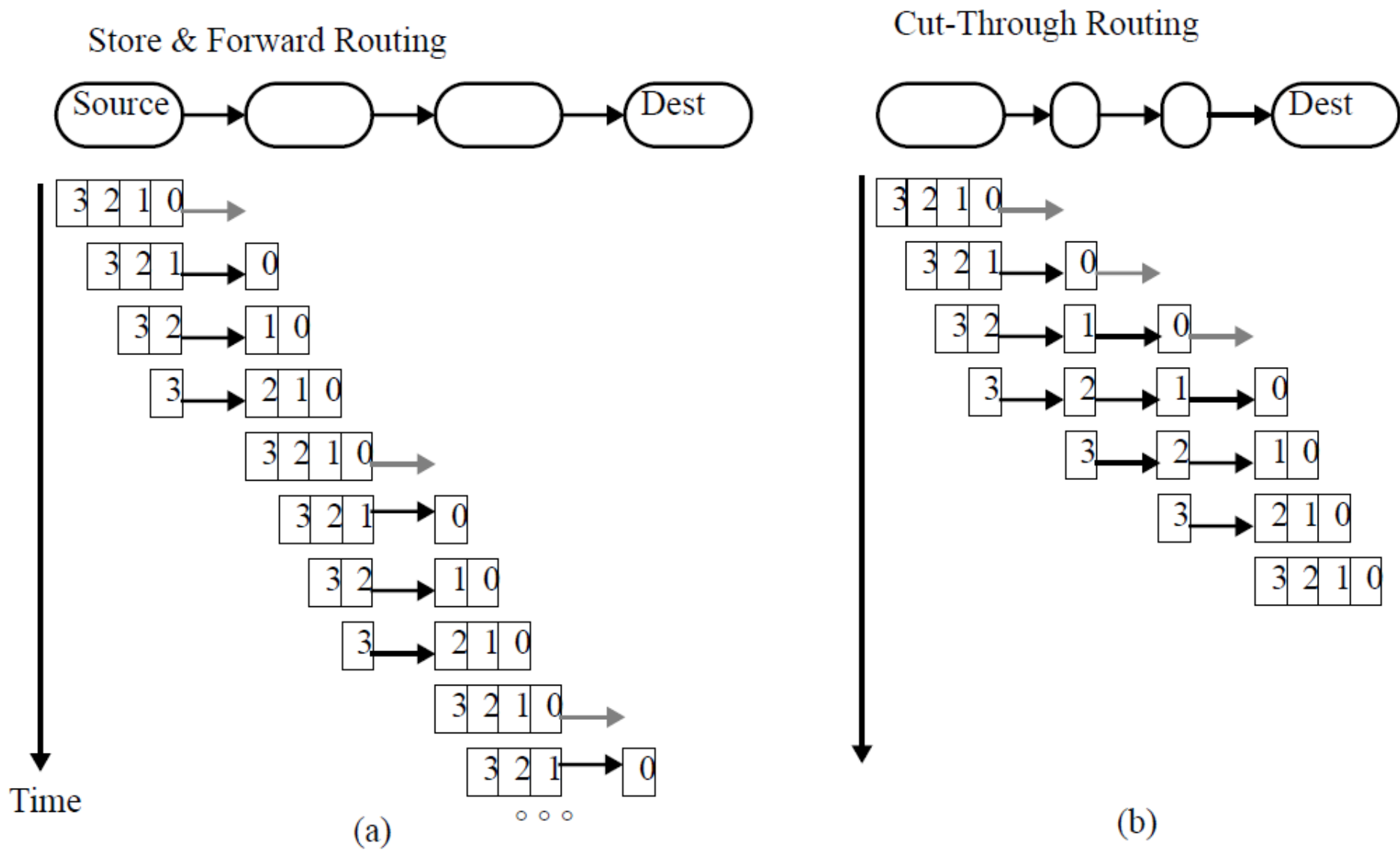


Figure 10-3 Store-and-forward vs. Cut-through routing

Hockney Model / Postal Model

- Communication time = $t_s + l.t_h + n/B$
 - Communicate in bulk t_s high: multiple small messages
 - Communicate over fewer hops $l.t_h$ high: large number of hops
 - Communicate less volume n/B high: large messages

- Simplified model: $T_{\text{comm}} = L + n/B$

Q: What are the factors that may reduce the effective bandwidth?

(1) The three points mentioned above

(2) Network congestion (3) Network topology

Latency and Bandwidth – The Traffic Analogy

SOURCE: ECESSA



SOURCE: DAILYNEWS



A Realistic Cost Model

Effective communication time

- $L + n/B + \text{contention}$

Difficult to analyse (Real-time network traffic unpredictable – depends on other jobs). An analogy is shown in the figure.

Reduce communication time by reducing

- Overhead (per-hop time)
- Contention
- Simplified model (without congestion):
 $T_{\text{comm}} = L + n/B$
- Simplified model (with congestion):
 $T_{\text{comm_congest}} = L' + n/B'$



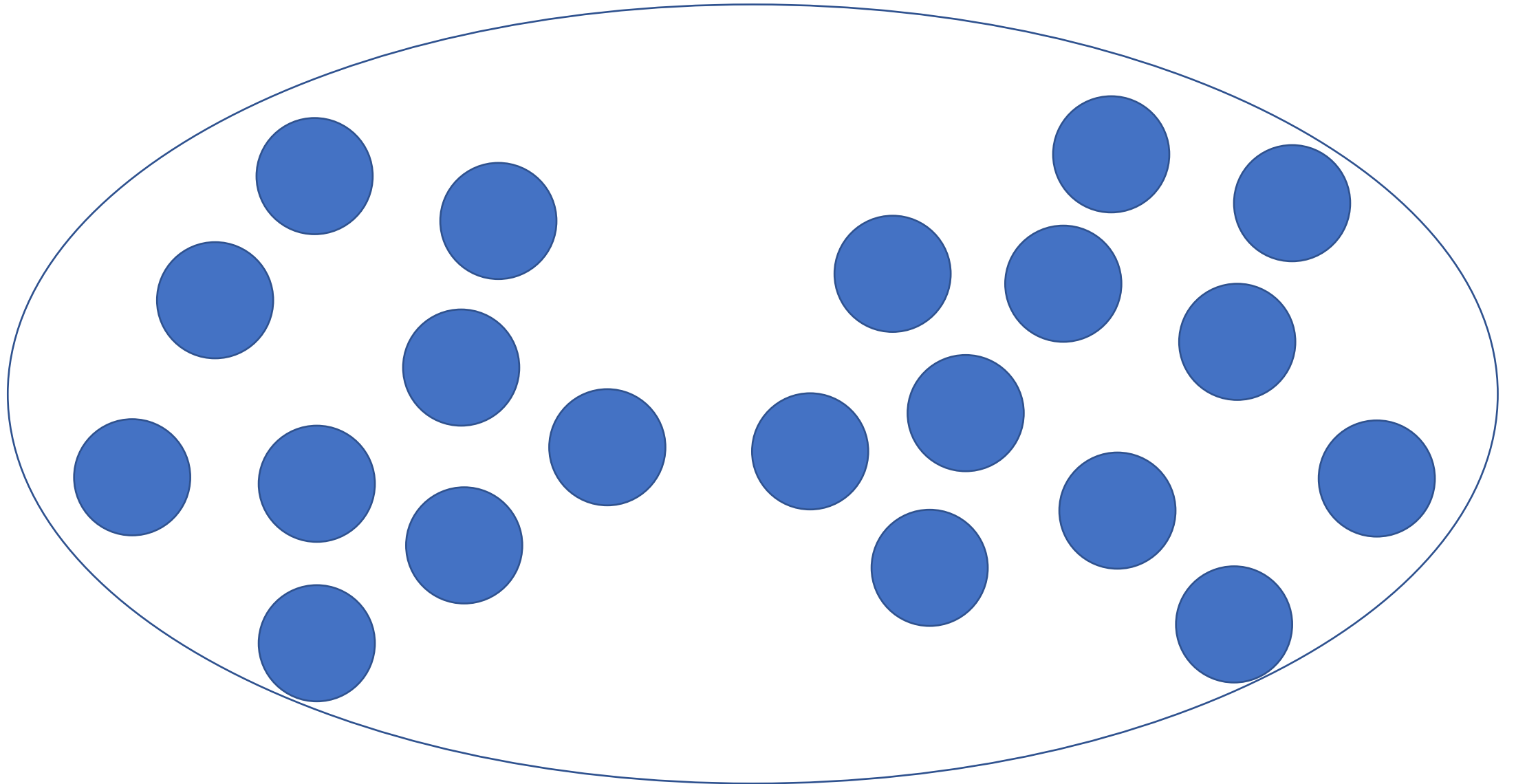
Communicator

- Object containing a group of processes
- Representative of communication domain
- Predefined:
 - `MPI_COMM_WORLD`
- Memory proportional to #processes in the group
- Several communication contexts may co-exist within a single communicator

Process Group

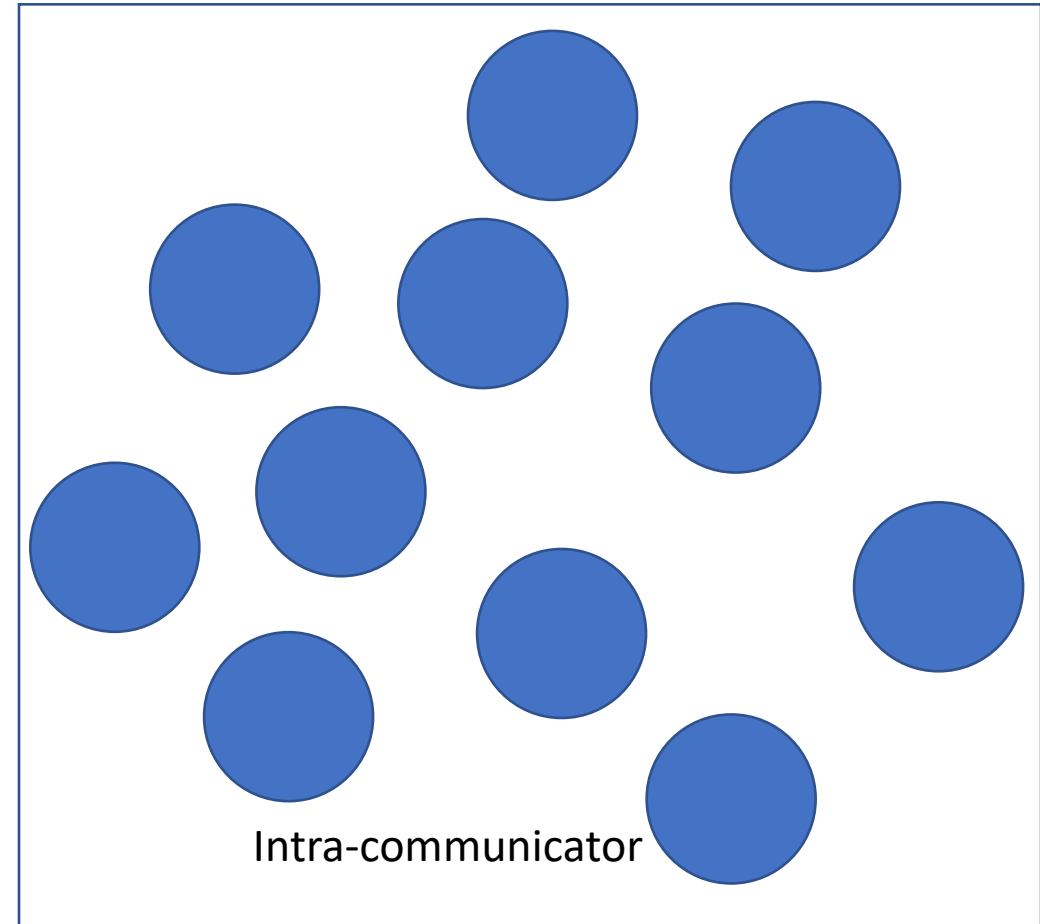
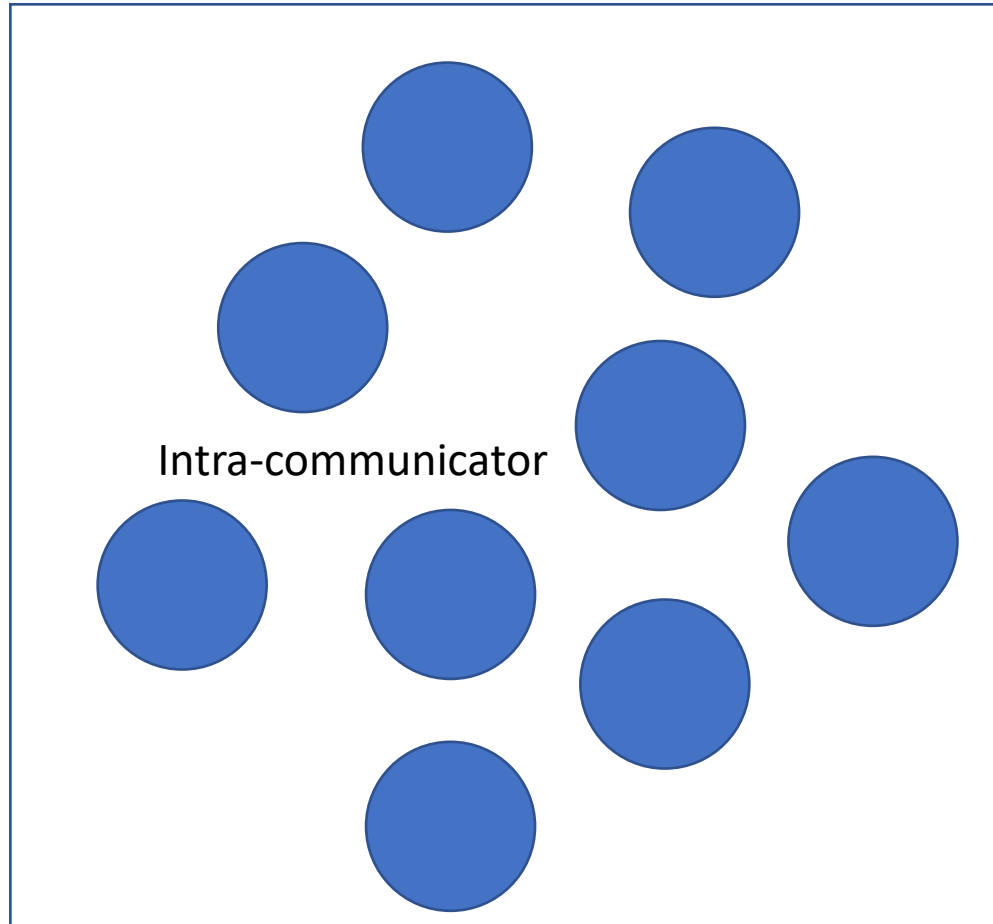
- Ordered set of processes
- Ranks are contiguous
- Base group – associated with MPI_COMM_WORLD
 - MPI_Comm_group (MPI_Comm comm, MPI_Group **group*)
- MPI_Group object
 - MPI_Group_rank, MPI_Group_size
- Operations: Unions and intersections of groups

MPI_COMM_WORLD



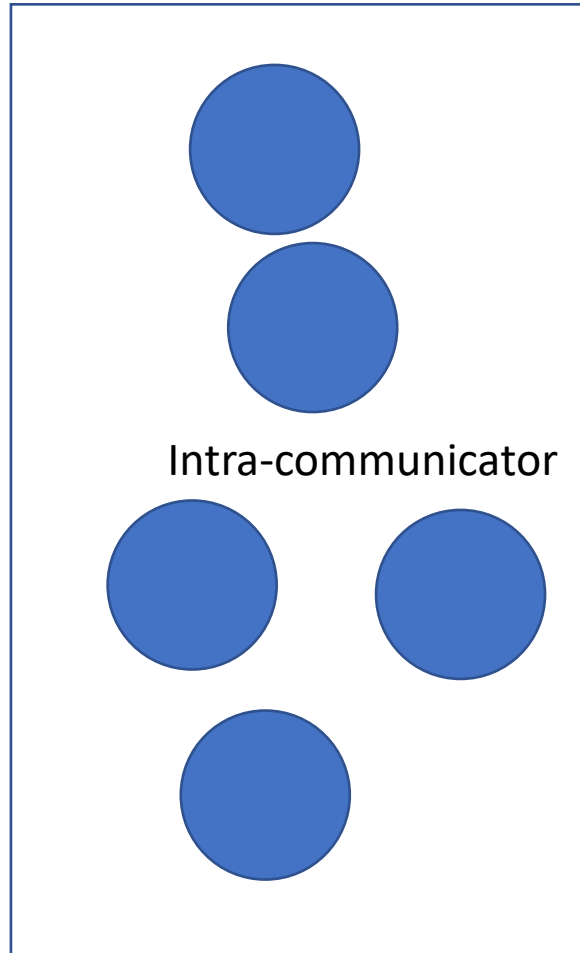
Intra-Communicators

- Logical subset
- Different contexts

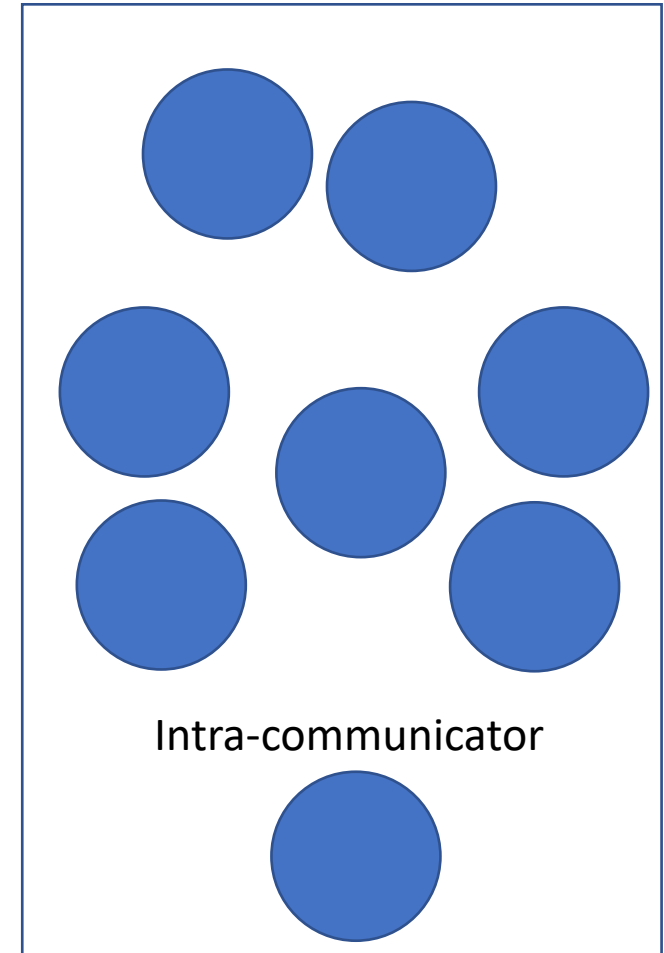


Intra-Communicators

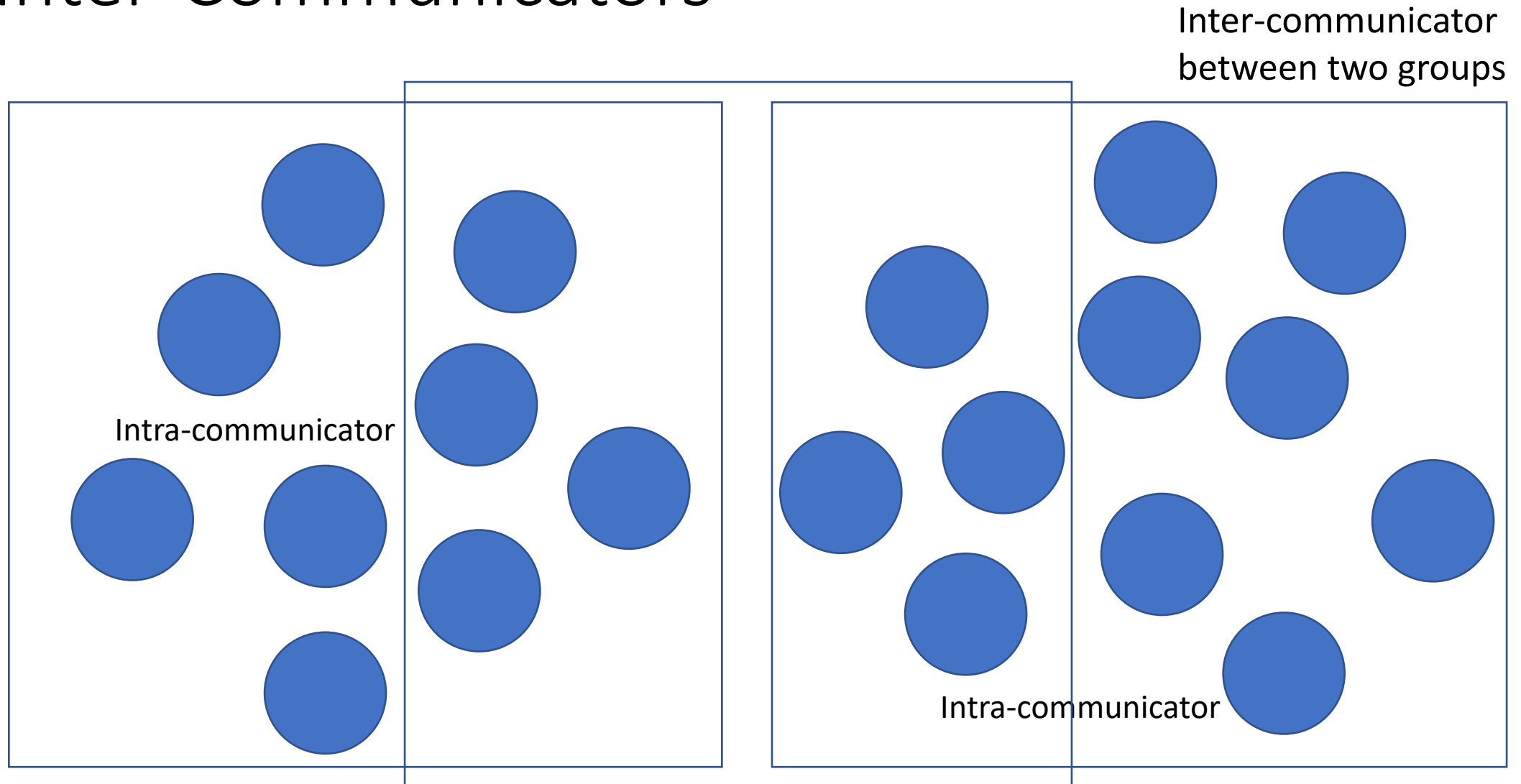
Group 1



Group 2



Inter-Communicators



MPI_COMM_SPLIT

```
MPI_Comm_split (  
    MPI_Comm oldcomm, int color, int key,  
    MPI_Comm *newcomm  
)
```

- Collective call (every process belonging to the oldcomm has to participate)
- Logically divides based on **color**
 - Processes of same color form a group
- Rank assignment based on key
- Output: newcomm

Code and Output

```
MPI_Comm_split (MPI_COMM_WORLD, color, myrank /*preserving original rank
order*/, &newcomm);

MPI_Comm_rank( newcomm, &newrank );
MPI_Comm_size( newcomm, &newsize );

MPI_Comm_free(&newcomm);
```

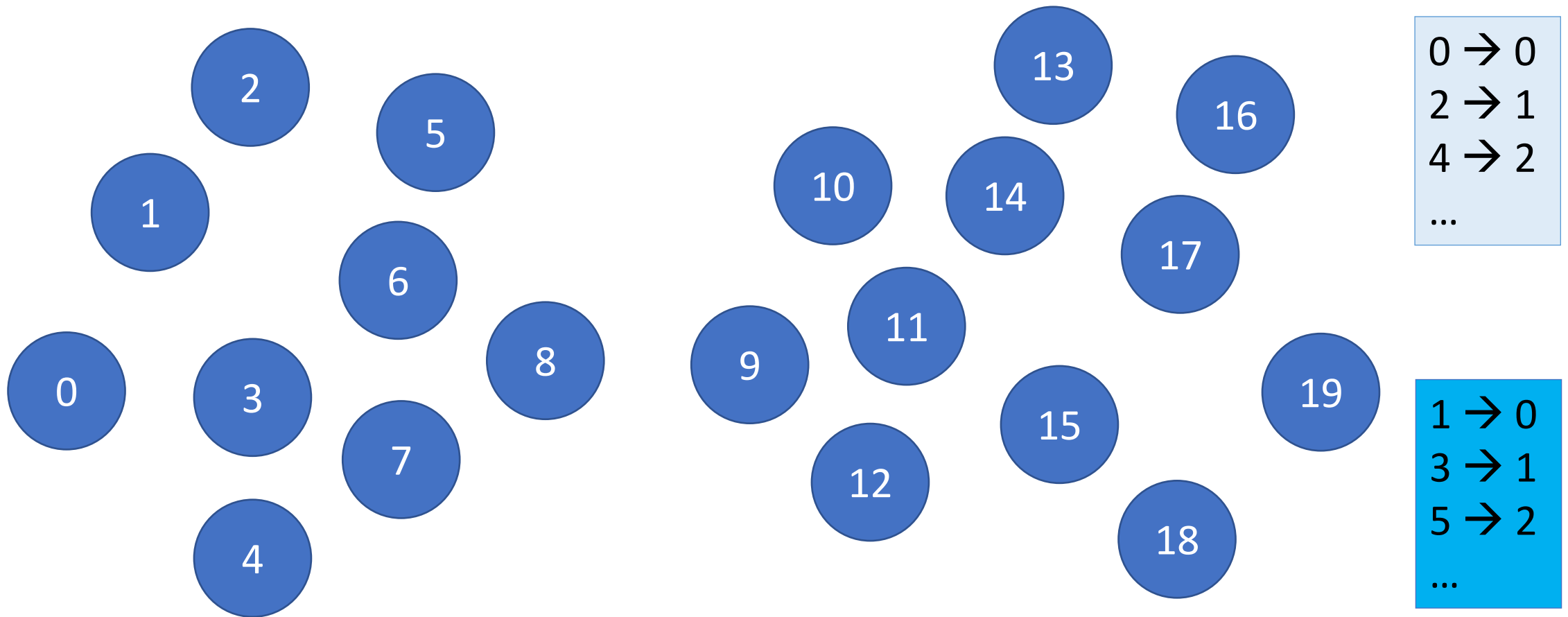
```
mpirun -np 4 ./split
Rank 0: new rank 0 of 2
Rank 1: new rank 0 of 2
Rank 2: new rank 1 of 2
Rank 3: new rank 1 of 2
```

Add `if (myrank)` before `MPI_Comm_`

```
class$ mpirun -np 4 ./split
Rank 0: new rank 569488416 of 5353
```

Incorrect code

Logical Subsets



How do you assign one color to odd processes and another color to even processes ?
 $\text{color} = \text{rank} \% 2$

MPI_Comm_split Example Code

Output
for P=8

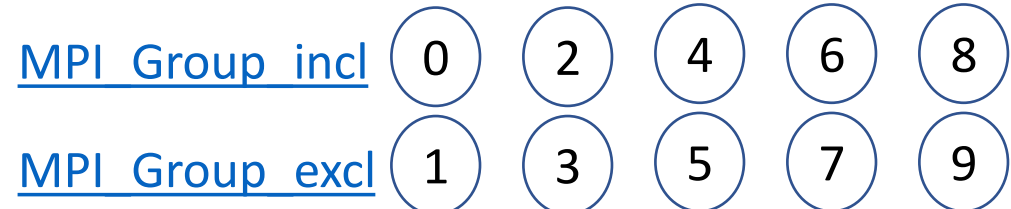
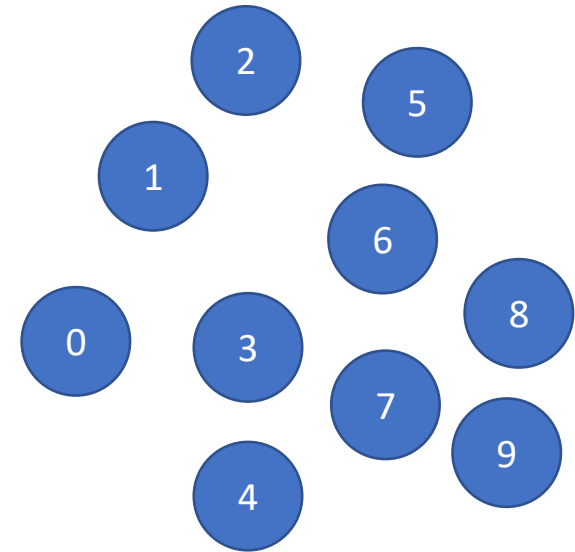
```
int newrank, newsiz, color = myrank%2;
MPI_Comm newcomm;
MPI_Comm_split (MPI_COMM_WORLD, color, myrank,
&newcomm);
MPI_Comm_rank (newcomm, &newrank);
MPI_Comm_size (newcomm, &newsiz);
printf ("%d: %d of %d\n", myrank, newrank,
newsiz);
MPI_Comm_free (&newcomm);
```

0: 0 of 4
1: 0 of 4
2: 1 of 4
3: 1 of 4
4: 2 of 4
5: 2 of 4
6: 3 of 4
7: 3 of 4

MPI_Group

```
MPI_Group_incl (  
    MPI_Group g_group,  
    int N, // size of the new group  
    const int ranks[],  
    MPI_Group *new_group);
```

```
MPI_Comm_create_group (  
    MPI_Comm comm,  
    MPI_Group new_group,  
    int tag,  
    MPI_Comm *new_comm);
```



MPI_Group Example Code

```
int ranks[] = {0,2,4,6,8}, N = 5;
```

```
// get the world group  
MPI_Group g_group;  
MPI_Comm_group (MPI_COMM_WORLD, &g_group);
```

```
// Construct a group containing even ranks in g_group
```

```
MPI_Group new_group;
```

```
MPI_Group_incl (g_group, N, ranks, &new_group);
```

```
// Create a new communicator based on the group
```

```
MPI_Comm new_comm;
```

```
MPI_Comm_create_group (MPI_COMM_WORLD, new_group, tag, &new_comm);
```

Code (group.c) – 2 groups: Odd and even ranks

```
int ranks[numranks/2], i, j=-1;

if (rank%2 == 0)
{
    for (i=0; i<numranks; i+=2)
        ranks[++j] = i;
}
else
for (i=1; i<numranks; i+=2)
    ranks[++j] = i;
```


MPI_Group_incl code (group.c)

```
// create a new group
MPI_Group new_group;
MPI_Group_incl (g_group, ranks_size, ranks, &new_group);

MPI_Comm new_comm;
MPI_Comm_create_group (MPI_COMM_WORLD, new_group, 100, &new_comm);

// size of new comm
int new_size, new_rank;
MPI_Comm_size (new_comm, &new_size);
MPI_Comm_rank (new_comm, &new_rank);
printf ("Old rank %d, new rank %d\n", rank, new_rank);
```

Homework

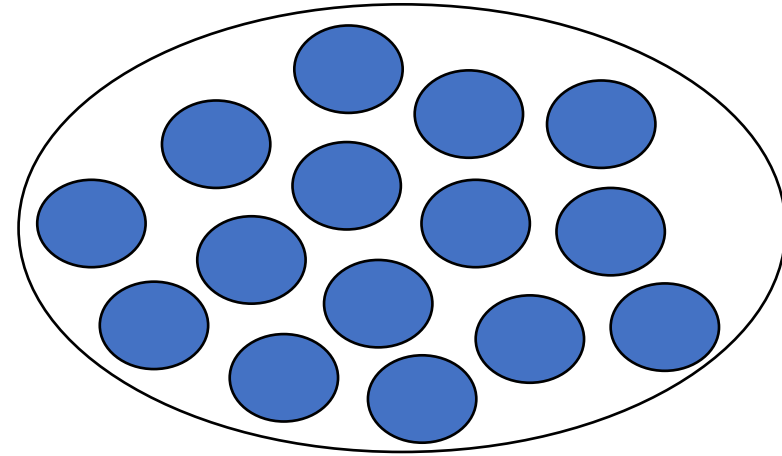
Q: Group every third process (2, 5, ...) and create a sub-communicator using

- `MPI_Comm_split`
- `MPI_Group_incl + MPI_Comm_create_group`

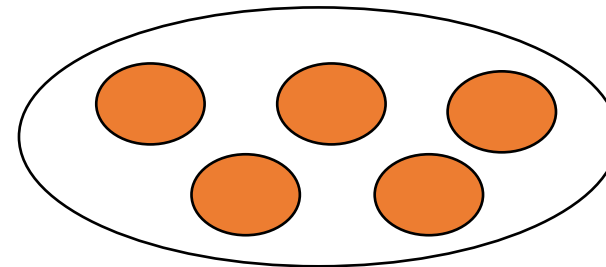
```
// get number of processes
// get my rank
// get the world group
// create new group ranks array
// create a new group and communicator
// get size of new comm and new rank
```

Usage of Sub-communicators

- Producer/client processes

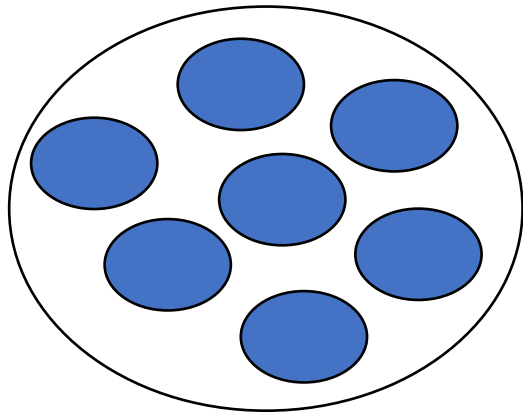


- Consumer/server processes

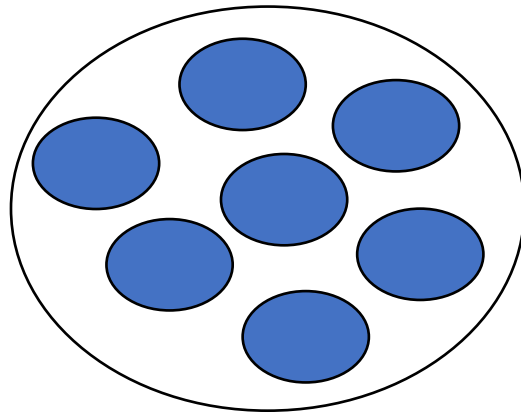
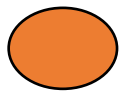


Producer-Consumer Example

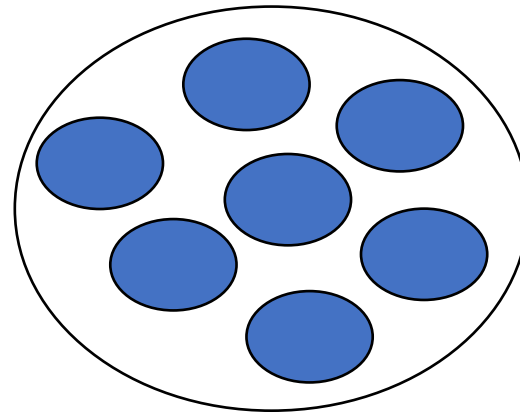
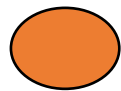
● Simulation
● Analysis/visualization



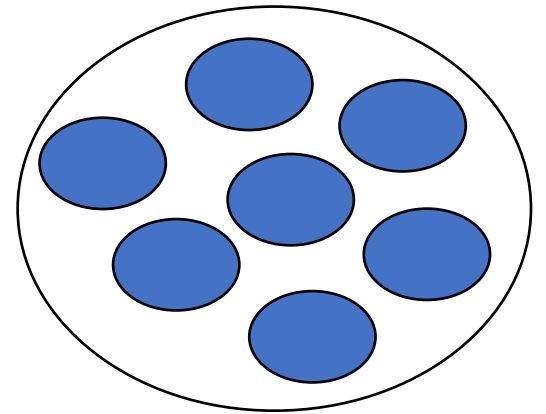
Many-to-one



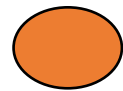
Many-to-one



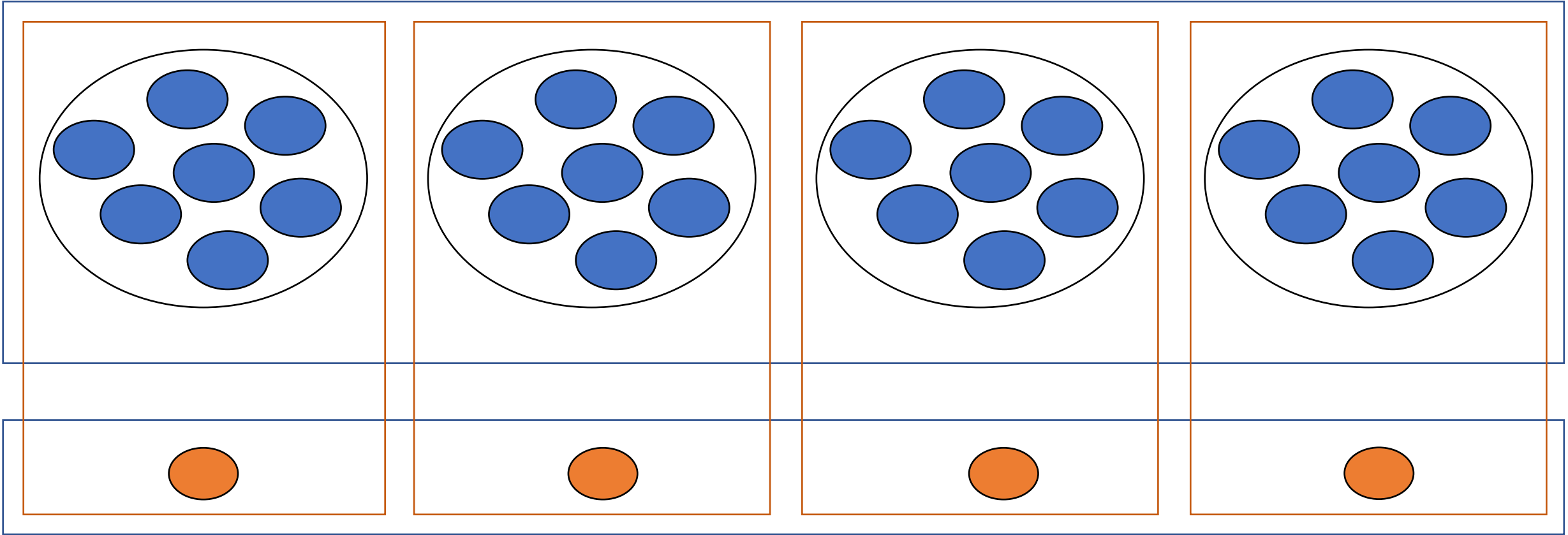
Many-to-one



Many-to-one



Producer-Consumer Example

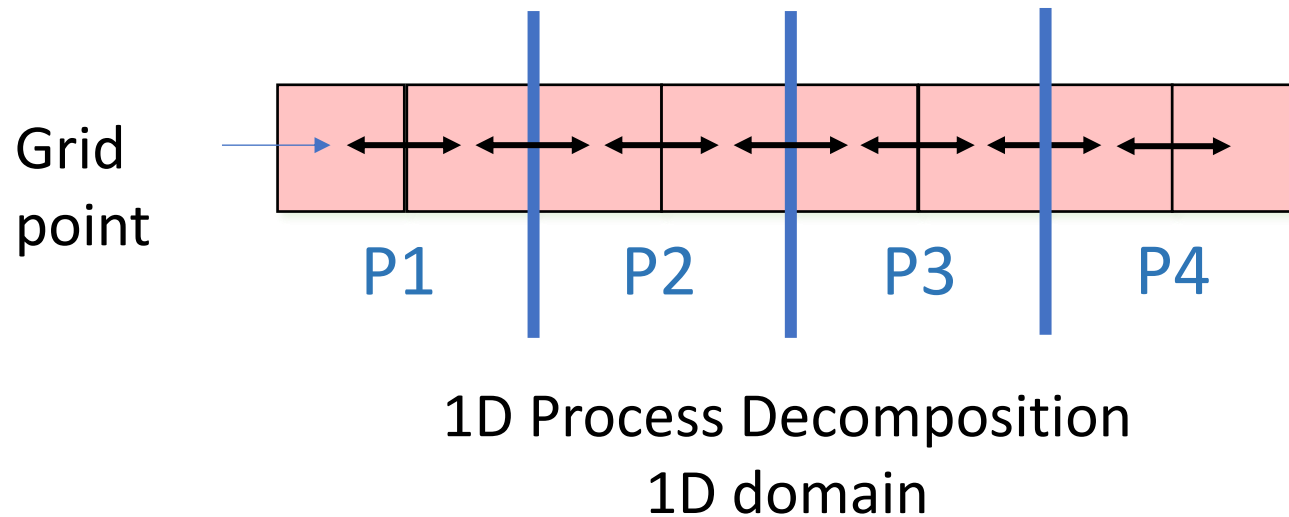


2 sub-communicators suffice: split 1 (blue and orange processes) and split 2 (orange rectangles for each of the four sub-groups)

Homework

- Write the code

1D Domain Decomposition



N grid points

P processes

N/P points per process

#Communications? 2

#Computations? N/P

Nearest
neighbor
communications

2 sends()
2 recvs()

Communication to computation ratio = $2P/N$